

End-to-End Workflow Report

1. Data Extraction (crawling the SQLite database)

The source data lives in *player_premier_league.db*. A Python script reads the `player_stats` table, selects the numeric columns, and stores a CSV file `player_stats_summary.csv` containing median, mean and population standard deviation for each statistic grouped by squad.

2. Data Analysis

- K-means clustering is evaluated for $k = 2 \dots 10$.
- Elbow curve (inertia) and Silhouette scores are plotted to decide the optimal k .
- PCA reduces the feature space to 2D and 3D; scatter plots `pca_2d.png` and `pca_3d.png` show the clusters.

3. Server Implementation (FastAPI)

A tiny FastAPI service is created (file `server.py`). It exposes two routes:

```
from fastapi import FastAPI from fastapi.responses import FileResponse, JSONResponse import pandas as pd import json app = FastAPI() @app.get('/stats') async def get_stats(): return FileResponse('player_stats_summary.csv', media_type='text/csv') @app.get('/clusters') async def get_clusters(): df = pd.read_csv('player_stats_with_clusters.csv') return JSONResponse(df.to_dict(orient='records'))
```

4. Client Implementation (CLI)

```
import requests import sys BASE = 'http://127.0.0.1:8000' if sys.argv[1] == 'stats': r = requests.get(f'{BASE}/stats') open('downloaded_stats.csv', 'wb').write(r.content) print('CSV saved as downloaded_stats.csv') elif sys.argv[1] == 'clusters': r = requests.get(f'{BASE}/clusters') print(r.json()) else: print('usage: client.py [stats|clusters]')
```

5. User Request Flow

When the user runs the client and asks for the plots, the client contacts the server. If any of the PNG files are missing, the server triggers the same logic used in the original script (`generate_kmeans_pca.py`) to recompute and store the missing images before responding. Thus the system is self-healing: the user never needs to manually delete/recreate the plots – the request itself causes the regeneration.